



UNIVERSIDAD TECNOLÓGICA DE PUEBLA INGENIERÍA EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

APLICACIÓN DE ESCRITORIO

REPORTE — SUBPRODUCTO NO. 3

**ASIGNATURA: APLICACIONES WEB
PROGRESIVAS**

**DOCENTE: MATHER XÓCHITL MENDOZA
PÍSCIL**

**ALUMNO: PÉREZ ALCÁNTARA ALEJANDRO
10° “D”**

FECHA: 21 DE SEPTIEMBRE DE 2026

ÍNDICE

1. Introducción.....	3
2. ¿Qué es una Aplicación de Escritorio?.....	3
2.1 Tipos de Aplicaciones de Escritorio.....	3
3. Ilustración: ¿Cómo funciona una Aplicación de Escritorio?	4
4. Características Principales	5
5. Ventajas y Desventajas	5
6. Comparativa: Escritorio vs. Web vs. Nativa vs. Híbrida	6
7. Tecnología Utilizada	7
8. Ejemplo de Funcionamiento: TaskDesk	8
9. Tabla Comparativa: Requisitos de una Aplicación de Escritorio vs. TaskDesk ..	11
10. Lista de Cotejo.....	12
11. Conclusión	13
12. URL de la Aplicación	13
Referencias.....	14

1. Introducción

A lo largo de esta materia he documentado los tres enfoques principales para construir software: en el Subproducto No. 1 documenté el desarrollo nativo, usando como ejemplo propio mi proyecto de reloj inteligente en Wear OS; en el Subproducto No. 2 documenté el desarrollo web, usando como ejemplo propio mi proyecto MediCore Professional System. En este tercer y último reporte cierro el panorama con el enfoque de aplicaciones de escritorio: programas que se instalan o ejecutan directamente sobre el sistema operativo de una computadora (Windows, macOS o Linux), sin depender de un navegador ni de una conexión a internet para funcionar.

A lo largo del documento defino el concepto de aplicación de escritorio, ilustro su funcionamiento mediante un diagrama, listo sus características principales, sus ventajas y desventajas, la comparo frente a los otros tres enfoques ya documentados (web, nativo e híbrido), documento la tecnología utilizada y, como evidencia propia, presento un programa que desarrollé para esta actividad: TaskDesk, un gestor de tareas personales en Python y Tkinter, completamente independiente y autocontenido. Cierro el documento con una tabla comparativa entre lo que exige la definición de una aplicación de escritorio y lo que realmente implementé, una lista de cotejo de autoevaluación y la URL del repositorio donde publiqué el código.

2. ¿Qué es una Aplicación de Escritorio?

Una aplicación de escritorio es un programa de software diseñado para instalarse y ejecutarse directamente sobre el sistema operativo de una computadora personal (Windows, macOS o Linux), en contraste con una aplicación web, que corre dentro de un navegador y depende de un servidor remoto. Una vez instalada —o, en el caso de un script interpretado como el mío, una vez que el intérprete correspondiente está disponible—, la aplicación de escritorio se ejecuta como un proceso local: toda la lógica del programa corre en la CPU y la memoria RAM del propio equipo, y sus datos normalmente se guardan en el sistema de archivos local, sin necesidad de conexión a internet.

Esto es exactamente lo opuesto a lo que documenté en el Subproducto No. 2: ahí expliqué cómo una aplicación web delega el procesamiento y el almacenamiento a un servidor remoto; aquí, una aplicación de escritorio hace ambas cosas en la propia máquina del usuario, lo que le da acceso más directo al hardware (sistema de archivos, impresoras, periféricos) y le permite funcionar sin depender de que haya servicio de internet.

2.1 Tipos de Aplicaciones de Escritorio

Identifiqué tres categorías principales de aplicaciones de escritorio conforme investigué para este reporte:

- **Nativas del sistema operativo:** compiladas específicamente para Windows (C#/ .NET, C++), macOS (Swift) o Linux (GTK/Qt); tienen el mejor rendimiento y la integración más profunda con el sistema, pero cada plataforma requiere su propio código.
- **Multiplataforma con runtime propio:** construidas con un framework que empaqueta su propio motor de renderizado, como Electron (JavaScript/HTML/CSS) o Qt; el mismo código corre en varios sistemas operativos a cambio de un ejecutable más pesado.
- **Scripts con interfaz gráfica sobre un intérprete:** como mi proyecto TaskDesk, escrito en Python con la librería estándar Tkinter; no requieren un compilador y corren en cualquier equipo que tenga el intérprete de Python instalado, ideal para herramientas internas y prototipos rápidos.

3. Ilustración: ¿Cómo funciona una Aplicación de Escritorio?

El siguiente diagrama ilustra el flujo general de una aplicación de escritorio: desde que el usuario ejecuta el programa, hasta que el sistema operativo carga la interfaz, el proceso corre localmente y el resultado se guarda y se muestra sin salir del propio equipo.

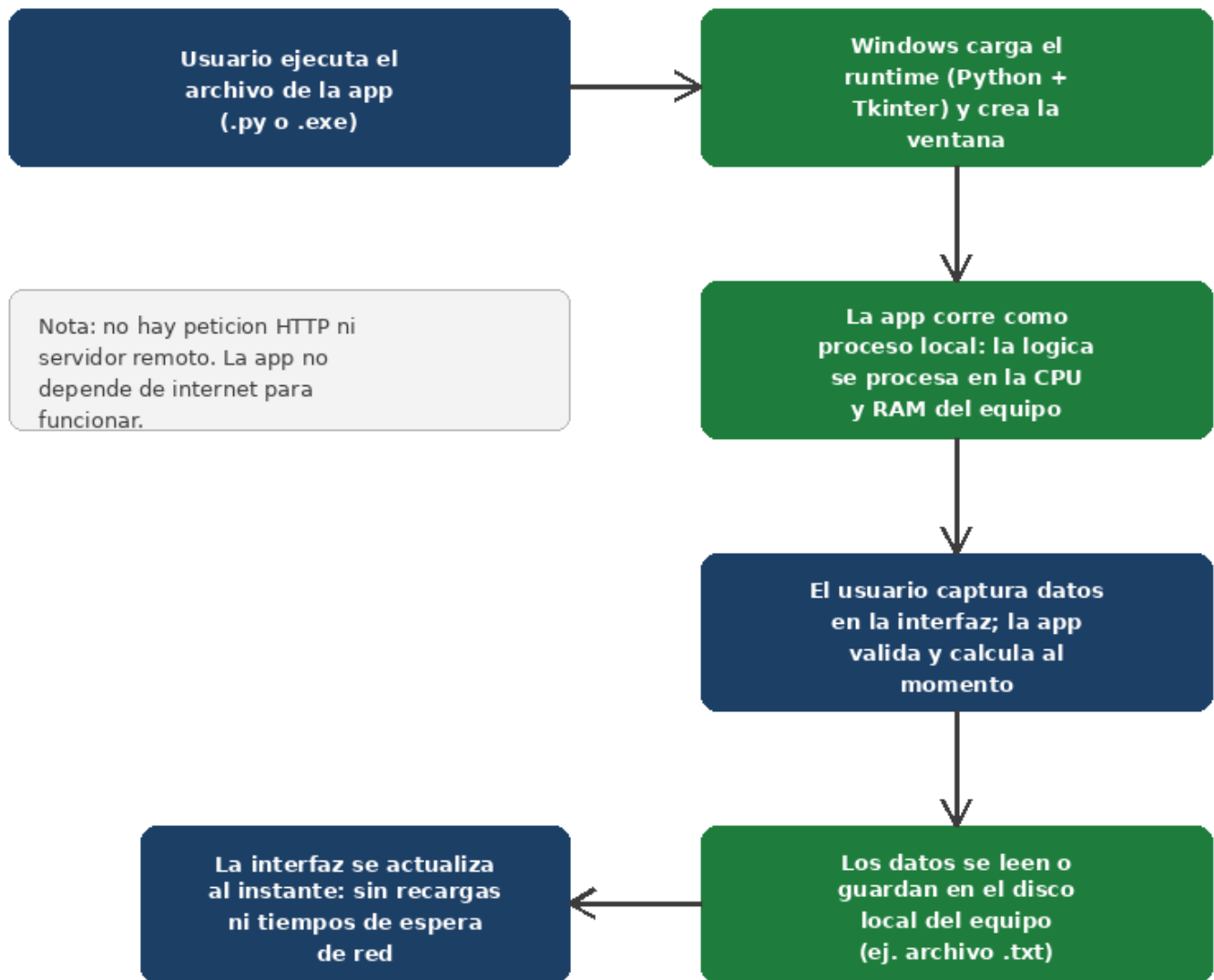


Figura 1. Flujo de funcionamiento de una aplicación de escritorio, desde que el usuario ejecuta el programa hasta que el resultado se guarda y se muestra en pantalla, todo dentro del propio equipo.

4. Características Principales

- **Requiere instalación o un intérprete disponible:** a diferencia de una app web, debe instalarse (o, como en mi caso, ejecutarse sobre un intérprete ya instalado) antes de poder usarse.
- **Ejecución como proceso local:** toda la lógica de negocio corre en la CPU y RAM del propio equipo, no en un servidor remoto.
- **Acceso directo al sistema de archivos y al hardware:** puede leer y escribir archivos locales, y acceder a periféricos (impresoras, cámaras, puertos) sin las limitaciones de un navegador.
- **Funciona sin conexión a internet:** al no depender de un servidor, puede operar completamente offline (salvo que ella misma implemente funciones de red).
- **Actualización manual o por instalador:** a diferencia de la actualización centralizada de una app web, cada equipo necesita recibir e instalar la nueva versión por separado.
- **Rendimiento e integración con el sistema operativo:** suele responder más rápido que una app web porque no depende de la latencia de red, y puede integrarse con notificaciones, el explorador de archivos o el menú de inicio del sistema operativo.

5. Ventajas y Desventajas

A partir de las características anteriores, resumo las principales ventajas y desventajas del desarrollo de escritorio, tal como hice en los Subproductos No. 1 y No. 2 para los enfoques nativo y web:

VENTAJAS	DESVENTAJAS
Funciona sin conexión a internet: todo el procesamiento y almacenamiento ocurre en el equipo.	Requiere instalación (o un intérprete/runtime) en cada equipo donde se use.
Acceso directo y completo al hardware y al sistema de archivos del equipo.	Actualizar la app implica distribuir e instalar una nueva versión en cada equipo, no hay actualización centralizada.
Mejor rendimiento y tiempos de respuesta al no depender de la latencia de red.	No es multiplataforma "gratis": una app nativa necesita adaptarse o recompilarse por sistema operativo.
Los datos sensibles pueden quedar solo en el equipo local, sin exponerse en la red.	Es más difícil de compartir o de que otra persona la use de inmediato, comparado con solo enviar una URL.

6. Comparativa: Escritorio vs. Web vs. Nativa vs. Híbrida

Para cerrar el panorama de los cuatro enfoques que documenté a lo largo de esta materia, comparo aquí el desarrollo de escritorio frente a los otros tres: web (Subproducto No. 2), nativo e híbrido (ambos documentados en el Subproducto No. 1).

CRITERIO	ESCRITORIO	WEB	NATIVA
Lenguaje	Python, C#, C++, Java (según SO)	HTML, CSS, JS (+ backend)	Swift/Kotlin (por plataforma)
Rendimiento	Alto	Medio-bajo	Alto
Acceso al hardware	Amplio (archivos, periféricos)	Limitado (APIs del navegador)	Completo
Instalación	Sí (o requiere runtime/intérprete)	No requiere, se accede por URL	Tienda oficial (App Store/Google Play)
Funciona sin internet	Sí	Muy limitado	Sí
Costo de desarrollo	Medio (1 código por SO o runtime propio)	Bajo (1 solo código)	Alto (2+ códigos independientes)
Ejemplos	TaskDesk, VS Code, Photoshop	MediCore, Google Docs, PWA	Spotify, mi app Wear OS

El desarrollo de escritorio ofrece el mejor acceso al hardware junto con el enfoque nativo, y la ventaja de funcionar completamente sin conexión a internet, algo que ni la web ni la híbrida logran del todo. A cambio, no comparte el bajo costo de mantenimiento de la web (cada equipo debe recibir la actualización por separado) y, a diferencia de una app nativa para móviles, no se distribuye a través de una tienda de aplicaciones. Elegí Python y Tkinter para TaskDesk precisamente por ese equilibrio: un lenguaje que ya conozco de mis prácticas, sin necesidad de compilar un ejecutable distinto por sistema operativo, ideal para una herramienta personal, ligera e independiente.

7. Tecnología Utilizada

Para TaskDesk utilicé la siguiente tecnología:

COMPONENTE	TECNOLOGÍA	FUNCIÓN
Lenguaje	Python 3	Lógica del programa: alta de tareas, prioridades y persistencia de datos
Interfaz gráfica	Tkinter (librería estándar de Python)	Ventanas, formularios, lista de tareas y botones de la aplicación
Almacenamiento	Archivo local en formato JSON (tareas.json)	Guarda cada tarea con su prioridad, fecha y estado, sin base de datos ni servidor
Empaquetado / distribución	Script .py ejecutado con el intérprete de Python	Permite correr la app en cualquier equipo con Python 3 instalado
Control de versiones	Git y GitHub	Publicación del código fuente como evidencia de la aplicación

Elegí Python y Tkinter porque Tkinter viene incluido en la instalación estándar de Python, sin necesidad de instalar dependencias adicionales ni empaquetar un ejecutable, lo que hace que cualquier compañero o el propio docente pueda ejecutar el programa con solo tener Python instalado. Para el almacenamiento, en vez de una base de datos, usé un archivo local en formato JSON: es la forma más simple de demostrar que una aplicación de escritorio puede guardar y leer información directamente del disco del equipo, sin depender de ningún servidor externo ni de conexión a internet.

8. Ejemplo de Funcionamiento: TaskDesk

Como ejemplo propio de aplicación de escritorio, desarrollé TaskDesk — Gestor de Tareas Personales: un programa independiente y autocontenido que permite agregar tareas con una prioridad (Alta, Media o Baja), marcarlas como completadas con doble clic, eliminarlas y consultar en todo momento cuántas están pendientes y cuántas completadas. La aplicación está construida en Python con Tkinter, corre como una ventana nativa de escritorio y guarda automáticamente cada tarea en un archivo local (tareas.json) en la misma carpeta del programa, por lo que la lista se conserva de una sesión a otra sin necesidad de ningún servidor ni conexión a internet.

A continuación, documento con capturas reales el funcionamiento de la aplicación, ejecutándola paso a paso: desde la ventana inicial vacía, la captura de nuevas tareas, hasta marcar una tarea como completada y eliminar otra de la lista.

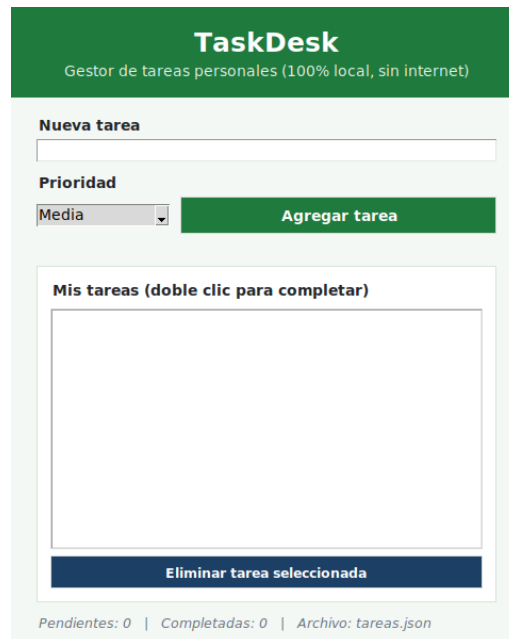


Figura 2. Ventana inicial de TaskDesk: lista de tareas vacía, lista para capturar la primera tarea.

The screenshot shows the TaskDesk application interface. At the top, a green header contains the title "TaskDesk" and the subtitle "Gestor de tareas personales (100% local, sin internet)". Below the header, there is a section for adding a new task. It includes a text input field with the value "Entregar reporte Subproducto No. 3", a dropdown menu for "Prioridad" set to "Alta", and a green button labeled "Agregar tarea". Below this, there is a section titled "Mis tareas (doble clic para completar)" which is currently empty. At the bottom of this section is a blue button labeled "Eliminar tarea seleccionada". At the very bottom of the application window, a status bar shows "Pendientes: 0", "Completadas: 0", and "Archivo: tareas.json".

Figura 3. Captura de una nueva tarea con su prioridad ("Entregar reporte Subproducto No. 3", prioridad Alta).

The screenshot shows the TaskDesk application interface after adding three tasks. The "Nueva tarea" section now has an empty text input field. The "Prioridad" dropdown menu is set to "Baja". The "Agregar tarea" button is still present. The "Mis tareas (doble clic para completar)" section now contains three tasks, each with a checkbox and its priority: "[] (Alta) Entregar reporte Subproducto No. 3", "[] (Media) Repasar para el examen de Negociacion", and "[] (Baja) Subir codigo a GitHub". The "Eliminar tarea seleccionada" button is still at the bottom of this section. The status bar at the bottom now shows "Pendientes: 3", "Completadas: 0", and "Archivo: tareas.json".

Figura 4. Tres tareas agregadas con distinta prioridad; el resumen inferior muestra 3 pendientes y 0 completadas.

The screenshot shows the TaskDesk application window. At the top is a green header with the title "TaskDesk" and the subtitle "Gestor de tareas personales (100% local, sin internet)". Below the header, there is a section for adding a new task. It includes a text input field labeled "Nueva tarea", a dropdown menu for "Prioridad" set to "Baja", and a green button labeled "Agregar tarea". Below this is a section titled "Mis tareas (doble clic para completar)". It contains a list of three tasks: "[] (Alta) Entregar reporte Subproducto No. 3", "[] (Media) Repasar para el examen de Negociacion", and "[X] (Baja) Subir codigo a GitHub". The third task is marked as completed with an "X" in the checkbox. Below the list is a dark blue button labeled "Eliminar tarea seleccionada". At the bottom of the window, a status bar shows "Pendientes: 2", "Completadas: 1", and "Archivo: tareas.json".

Figura 5. Una tarea marcada como completada con doble clic ("[X] Subir código a GitHub"); el resumen se actualiza a 1 completada.

This screenshot shows the TaskDesk application after a task has been deleted. The interface is identical to the previous one, but the status bar at the bottom now shows "Pendientes: 1" instead of 2. The task list in the "Mis tareas" section remains the same: "[] (Alta) Entregar reporte Subproducto No. 3", "[] (Media) Repasar para el examen de Negociacion", and "[X] (Baja) Subir codigo a GitHub". The "Agregar tarea" button is still present and functional.

Figura 6. Después de eliminar una tarea de la lista: el cambio queda guardado de inmediato en tareas.json, en el propio equipo y sin conexión a internet.

9. Tabla Comparativa: Requisitos de una Aplicación de Escritorio vs. TaskDesk

De la misma forma en que en los Subproductos No. 1 y No. 2 comparé mis proyectos contra lo que exige cada definición, aquí reviso, criterio por criterio, si TaskDesk cumple con lo que se espera de una aplicación de escritorio real:

CRITERIO DE APP DE ESCRITORIO	QUÉ EXIGE LA DEFINICIÓN	¿CUMPLE?	EVIDENCIA EN TASKDESK
Ejecución local	Debe correr como proceso del sistema operativo, no en un navegador	Sí	Se ejecuta con "python taskdesk.py", abre una ventana nativa de Tkinter
No depende de un servidor	La lógica debe procesarse en el propio equipo	Sí	Toda la lógica (agregar, completar, eliminar tareas) corre en el mismo proceso Python, sin llamadas de red
Funciona sin internet	Debe operar sin conexión	Sí	No usa ninguna librería de red; corre igual con el Wi-Fi apagado
Interfaz gráfica de escritorio	Debe tener ventanas, formularios y controles nativos	Sí	Ventana Tkinter con campos, lista, combobox y botones (ver Figuras 2 a 6)
Acceso al sistema de archivos local	Debe poder leer/escribir archivos del equipo	Sí	Guarda y relee todas las tareas en tareas.json, en la misma carpeta del programa
Validación de datos	Debe validar la entrada del usuario	Sí	Rechaza agregar una tarea si el campo de descripción está vacío
Instalación / distribución	Debe poder instalarse o distribuirse a otro equipo	Parcial	Se distribuye como script .py; requiere tener Python instalado (no se generó un .exe)
Actualización centralizada	Un cambio en el servidor debe reflejarse para todos de inmediato	No	Cada equipo debe recibir el archivo actualizado manualmente; es la limitación típica de una app de escritorio frente a una web

De los ocho criterios revisados, TaskDesk cumple con seis de forma completa y uno de forma parcial (la distribución, ya que se comparte como script en vez de un instalador .exe empaquetado), y solo no cumple con la actualización centralizada, que por definición es una ventaja exclusiva del enfoque web que documenté en el Subproducto No. 2. Esa diferencia no es una falla de implementación, sino precisamente la característica que distingue a una aplicación de escritorio de una aplicación web.

10. Lista de Cotejo

Reviso, a manera de autoevaluación, que el documento cumpla con todos los puntos solicitados para esta actividad:

ELEMENTO SOLICITADO	CUMPLE	OBS.
Portada con formato oficial UTP	Sí	
Índice / tabla de contenido	Sí	
Introducción	Sí	
Definición de qué es una aplicación de escritorio	Sí	
Ilustración del funcionamiento (diagrama)	Sí	
Características principales	Sí	
Ventajas y desventajas	Sí	
Comparativa (escritorio vs. web vs. nativa vs. híbrida)	Sí	
Tecnología utilizada	Sí	
Ejemplo de mi propia aplicación de escritorio (TaskDesk)	Sí	
Imágenes de mi aplicación (evidencia propia)	Sí	5 capturas reales de la app en ejecución
Tabla comparativa: requisitos vs. mi aplicación	Sí	
URL de la aplicación (repositorio)	Sí	
Conclusión	Sí	
Referencias	Sí	

11. Conclusión

Las aplicaciones de escritorio representan el enfoque más directo para aprovechar el hardware y el sistema de archivos de un equipo, y la única de las cuatro que documenté en esta materia que puede funcionar por completo sin conexión a internet. A cambio, dependen de que cada equipo tenga instalado el programa (o, en mi caso, el intérprete de Python), y no cuentan con la actualización centralizada que sí tiene una aplicación web como MediCore Professional System.

Desarrollar TaskDesk me permitió comprobar en la práctica esa diferencia: mientras que en el Subproducto No. 2 documenté cómo MediCore procesa todo en un servidor remoto en Railway, aquí construí una herramienta completamente independiente, que guarda y gestiona información sin salir del equipo del usuario ni depender de ningún servicio externo.

Comparar en los tres subproductos el mismo tipo de análisis —definición, características, ventajas y desventajas, comparativa y tabla de cumplimiento— pero aplicado primero a un proyecto nativo (Wear OS), luego a uno web (MediCore) y finalmente a uno de escritorio (TaskDesk), me deja un panorama completo de que la elección entre estos enfoques no es una cuestión de cuál es "mejor" en abstracto, sino de qué tanto pesa cada criterio —rendimiento, acceso al hardware, costo, mantenimiento y dependencia de internet— en el proyecto específico que se está construyendo.

12. URL de la Aplicación

Como evidencia del ejemplo presentado (TaskDesk), se listan sus datos de acceso. A diferencia de una aplicación web, una aplicación de escritorio no se "visita" desde una URL pública: la evidencia de su funcionamiento es el propio código fuente publicado, junto con las capturas reales incluidas en la sección 8 de este reporte.

- **Repositorio de código fuente:** <https://github.com/AlexAP1115web/APA-AplicacionDeEscritorio.git>

PASOS: al clonar o descargar el repositorio, ejecutar en una terminal con Python 3 instalado:

```
python taskdesk.py
```

La aplicación no requiere instalar dependencias adicionales, ya que Tkinter viene incluido en la instalación estándar de Python.

Referencias

- Microsoft. (2026). Introducción al desarrollo de aplicaciones de escritorio en Windows. <https://learn.microsoft.com/es-mx/windows/apps/desktop/>
- Python Software Foundation. (2026). tkinter — Python interface to Tcl/Tk. <https://docs.python.org/es/3/library/tkinter.html>
- Real Python. (2026). Python GUI Programming With Tkinter. <https://realpython.com/python-gui-tkinter/>
- Electron. (2026). Electron Documentation. <https://www.electronjs.org/docs/latest>
- Git. (2026). Git Documentation. <https://git-scm.com/doc>
- GitHub Docs. (2026). Uploading a project to GitHub. <https://docs.github.com/es/repositories>
- MDN Web Docs. (2026). Progressive web apps (PWAs). https://developer.mozilla.org/es/docs/Web/Progressive_web_apps